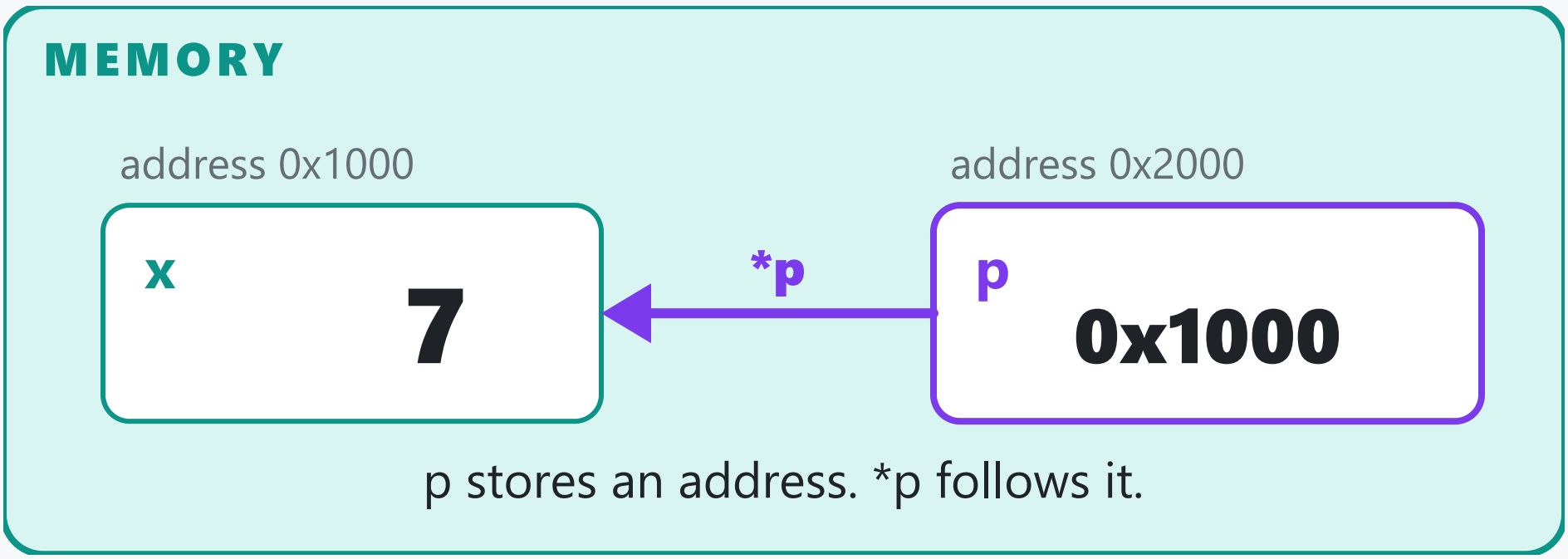
*p means: go to the address stored in p. Then use what is there.

```
int x = 7;
int* p = &x;
int y = *p;   // 7
```

That is dereferencing. The star appears in two jobs, and this is the second one.

swipe →

What is in memory



`&x` is the address of `x`. A pointer is just a variable that holds one.

Read it, write it

```
int y = *p;     // read: y is 7
*p = 9;          // write: x is now 9
```

`*p` **is** `x`, not a copy of it.

Assign to `*p` and you assign to `x`.

I ran it: `y` stayed 7, and `x` became 9.

Pointer to a struct

```
struct Player { int hp; };

Player player;
Player* q = &player;

(*q).hp = 20;
q->hp = 20;      // the same thing
```

`->` is a shortcut for "dereference, then pick a member".

The parentheses in `(*q).hp` are needed because `.` binds tighter than `*`.

Four expressions, four meanings

WRITE	IT IS
`x`	the value, 7
`&x`	the address of `x`
`p`	the address stored in `p`
`*p`	the value at that address, 7

`&p` is something else again: the address of the pointer itself.

When it goes wrong

THE POINTER IS	DEREFERENCING IT
`nullptr`	**undefined behaviour**
uninitialized	**undefined behaviour**
dangling (object gone)	**undefined behaviour**

It often crashes. It sometimes seems to work. Neither is guaranteed.

Where do you still need a raw *?